# Quest 8: The Entgineering

Lord of the Wings · shadows on a hexagonal field, in six directions, while the Ents grow

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

**In one sentence:** energy sweeps across a hexagonal field of Ents (heights 1–9) in one direction; each Ent casts a shadow that drops by 1 per hex, and you count the Ents that stick out above the shadow reaching them.
**Input:** a hex grid drawn as text: digits are Ents, dots only keep the layout.
**Output:** the number of Ents that receive energy (one unit each).
**What changes between parts:** Part I uses one direction (↘). Part II adds up all six hex directions on the same field. Part III makes the field change: lit Ents grow, height-9 Ents leave as warriors and come back as seeds, and you run the six-direction cycle until 100,000 warriors have left.

## 1. The story

With Sarumonk gone and the team out of the cage, the poisoned food is still a problem, so the hero turns to the Ents, who don't eat bread anyway. They offer to grow an army of wooden warriors on hexagonal growing fields, but first they need to know how much energy a field uses: a single test round, then a full circuit of the moving energy source, and finally the whole growing process.

## 2. Understanding the question

### Reading the hex grid

The field is a hexagon squeezed into a text grid. Ents sit on every other character, and each row is shifted by one, so the six neighbours of the Ent at (row *r*, column *c*) are:

| Direction | ↘ | ↙ | ← | ↖ | ↗ | → |
|---|---|---|---|---|---|---|
| Next hex | (r+1, c+1) | (r+1, c−1) | (r, c−2) | (r−1, c−1) | (r−1, c+1) | (r, c+2) |

Left and right move **two** columns, because the character in between is a layout dot. The diagonals move one row and one column.

### Shadows along one line

Energy travels in straight lines in the given direction. Walk along one line, from where the energy enters:

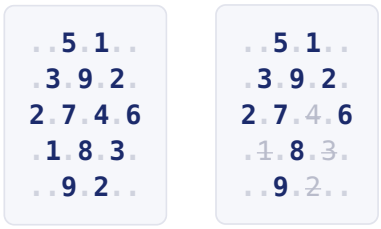
- An Ent of height *h* casts a shadow of *h* − 1 on the next hex, *h* − 2 on the one after, and so on.
- Every Ent casts its shadow, **even if it is itself shaded**. Overlapping shadows don't add up: the highest one wins.
- An Ent receives energy only if it is **taller** than the shadow reaching it. Equal height means shaded.

So a useful running value is the **"skyline"** $S$ at each hex: the higher of the Ent's own height and the shadow arriving there. The shadow arriving at the next hex is $S - 1$. An Ent of height $h$ is lit exactly when $h > S_{prev} - 1$, i.e. when $h \geq S_{prev}$.

### Extra examples of my own (one line each, worked by hand)

| Heights along the line | Skyline $S$ | Lit | Why it's interesting |
|---|---|---|---|
| 6, 5, 4, 7, 1 | 6, 5, 4, 7, 6 | 6, 7 (2 Ents) | The 5 and the 4 are exactly as tall as the shadow (5, then 4), and a tie means shaded. |
| 3, 9, 9 | 3, 9, 9 | all 3 | The second 9 only gets a shadow of 8, so it is lit. |
| 9, 1, 1, 1, 1, 1, 1, 1, 1 | 9, 8, 7, …, 1 | only the 9 | A 9 shades everything up to 8 hexes away. |

And a whole small field (5 rows, 14 Ents), run through a copy of the Part I code. The lit ones are bold, the shaded ones are struck through:




before ↘: 10 Ents lit

For example, in the ↘ line 2 → 1 → 9 (rows 2–4) the 1 is shaded by a shadow of exactly 1, and the 9 behind it is lit. In the line 5 → 9 → 4 → 3 (rows 0–3), the 9 shades the 4 (shadow 8) and the 3 (shadow 7). Running the same field through a copy of the Part II code gives 10, 8, 10, 7, 10 and 9 for the six directions: **54** in total.

### Shared code: reading the field

```
1  import { readFileSync } from "node:fs";
2
3  const isSample = process.argv.includes("--sample");
4  const file = isSample ? "sample1.txt" : "input1.txt";
5  const INPUTS: string[][] = readFileSync(file, "utf8")
6    .trim()
7    .split("\n")
8    .map((i) => i.split(""));
```

q8/part1.ts, lines 1–8

| Lines | What happens |
|---|---|
| 3–4 | `--sample` picks the sample file; otherwise the real input. |
| 5–8 | `INPUTS` is a 2D array of single characters, digits and dots, exactly as drawn. No conversion to numbers yet: the code keeps working with strings, and calls `Number(...)` only when it compares. |

## 3. Part I: one test round (↘)

### The question

Energy comes from the upper left and travels ↘. Count the Ents that receive it.

### Worked example (sample)




the sample field (61 Ents)




↘: 32 lit, matching the puzzle

The first line the puzzle shows is 4, 9, 7, 9, 1 (the column-0 Ent and the four hexes down-right of it): the 4 and both 9s are lit, the 7 and the 1 are shaded. By contrast, the line starting at the top Ent 1 (row 0, column 4) reads 1, 2, 3, 4, 5, 6, 6, 8, 8: it never goes down by more than the shadow allows, so all nine are lit.

**Step-by-step solution**

1. Read the field as a grid of characters.
2. Go through the grid top to bottom, left to right. For every Ent, look at the hex up-left of it: if there is none, its skyline is its own height; otherwise its skyline is the higher of its own height and (that hex's skyline – 1).
3. Go through again: an Ent whose height is below the up-left hex's skyline is shaded.
4. Count the Ents that are not shaded.

## My solution

```ts
const HEIGHTS: string[][] = (
  JSON.parse(JSON.stringify(INPUTS)) as string[][]
).map((i) => i.fill("."));

const getHeight = (row: number, col: number) => {
  if (INPUTS[row - 1]![col - 1] === ".") return INPUTS[row]![col] || ".";

  const currentHeight = Number(INPUTS[row]![col]);
  const prevHeight = Number(HEIGHTS[row - 1]![col - 1]);
  let height = 0;

  if (currentHeight > prevHeight - 1) height = currentHeight;
  else height = prevHeight - 1;

  return String(height);
};

for (let row = 0; row < INPUTS.length; row++) {
  for (let col = 0; col < INPUTS[0]!.length; col++) {
    if (INPUTS[row]![col] === ".") continue;

    if (row === 0 || col === 0) HEIGHTS[row]![col] = INPUTS[row]![col] || ".";
    else HEIGHTS[row]![col] = getHeight(row, col);
  }
}

for (let row = 0; row < INPUTS.length; row++) {
  for (let col = 0; col < INPUTS[0]!.length; col++) {
    if (
      row === 0 ||
      col === 0 ||
      INPUTS[row]![col] === "." ||
      HEIGHTS[row - 1]![col - 1] === "."
    )
      continue;

    if (Number(INPUTS[row]![col]) < Number(HEIGHTS[row - 1]![col - 1]))
      INPUTS[row]![col] = ".";
  }
}

let energyRequired = 0;

INPUTS.forEach((row) => {
  row.forEach((col) => {
    if (col !== ".") energyRequired++;
  });
});

console.log(energyRequired);
```

q8/part1.ts, lines 10–59

| Lines | What happens |
|---|---|
| 10–12 | `HEIGHTS` is a grid of the same size, filled with dots. The `JSON.parse(JSON.stringify(...))` makes a **deep copy**, so `fill(".")` doesn't wipe `INPUTS` too. |
| 27–34 | First pass, top to bottom and left to right. For every Ent, store its **skyline** value (the higher of its own height and the shadow arriving) in `HEIGHTS`. Because the previous hex is (row – 1, col – 1), it is always computed before the current one. |
| 31 | In row 0 or column 0 there is no previous hex, so the skyline is just the Ent's height. |
| 14–25 | `getHeight`: if the previous hex is a layout dot (outside the hexagon), the line starts here and the skyline is the own height (line 15). Otherwise it is `max(own, prev - 1)`, written out as an `if` (lines 21–22). |
| 36–49 | Second pass: an Ent is shaded when its height is **below** the previous hex's skyline (line 46). That's the same as "not taller than the shadow": $h < S \Leftrightarrow h \leq S - 1$ for whole numbers. Shaded Ents are overwritten with a dot. |
| 38–44 | Skip edge Ents and Ents whose previous hex is a layout dot: they are first in their line and always lit. |
| 51–59 | Count what is left that isn't a dot. |

### Trace: the first line of the sample (from an instrumented copy)

| (row, col) | Ent | `HEIGHTS` (skyline) | Line 46: Ent < previous skyline? |
|---|---|---|---|
| (4, 0) | 4 | 4 | lit (first in its line) |
| (5, 1) | 9 | 9 | lit: 9 is not below 4 |
| (6, 2) | 7 | 8 | **shaded**: 7 < 9 (the shadow reaching it is 8) |
| (7, 3) | 9 | 9 | lit: 9 is not below 8 |
| (8, 4) | 1 | 8 | **shaded**: 1 < 9 |

The full `HEIGHTS` grid after the first pass, next to the result (the skyline is never below the Ent itself):

```
....1.9.6.1.7....
...9.2.8.5.8.6...
..7.8.3.7.4.7.5..
.4.6.7.4.6.3.8.9.
4.5.5.6.5.8.9.7.9
.9.5.4.5.6.7.8.9.
..8.7.6.4.6.6.7..
...9.6.8.3.8.5...
....8.6.7.2.8....
```

`HEIGHTS` after lines 27–34

```
....1.9.6.1.7....
...9.2.3.3.8.1...
..7.8.3.5.1.4.3..
.4.6.7.4.1.1.8.9.
4.5.3.6.5.8.9.4.9
.9.5.1.5.6.2.8.9.
..7.7.6.4.6.2.4..
...9.3.8.3.8.4...
....1.6.6.2.8....
```

`INPUTS` after lines 36–49

> **Two passes are needed.** The first pass needs the original heights of the shaded Ents, because a shaded Ent still casts a shadow. If line 47 ran inside the first loop, a shaded 7 would turn into a dot and the next hex would see the line as starting fresh. Keeping the skyline in its own grid avoids that.

> **Small things:** the `|| "."` on lines 15 and 31 never does anything, because both are only reached for cells that hold a digit. Also, a dot in front of an Ent is treated as "the line starts here". That's right for a solid hexagon, but if a field had an empty hex *inside* it, a shadow ought to pass over the gap (dropping by 1) instead of stopping. Your inputs have no such gaps (checked), so this never matters.

## 4. Part II: a full circuit of six directions

### The question

The same field is lit from all six directions in turn (↘, ↙, ←, ↖, ↗, →). Add up the lit Ents of the six rounds.

### Worked example (sample)

| Round | Direction | Energy |
|---|---|---|
| 1 | ↘ | 32 |
| 2 | ↙ | 32 |
| 3 | ← | 29 |
| 4 | ↖ | 32 |
| 5 | ↗ | 30 |
| 6 | → | 32 |
| | **total** | **187** |

On my small field from section 2 the six rounds give 10 + 8 + 10 + 7 + 10 + 9 = **54**.

### The key idea: flip the field instead of writing six versions

Mirroring the grid doesn't change which Ents are lit, only where they are drawn. So instead of six shadow functions, your code writes one function that only knows "the previous hex is `top` rows up and `left` columns left", and flips the grid so the wanted direction becomes either ↘ (`top = 1, left = 1`) or → (`top = 0, left = 2`):

| Direction | Step in the original grid | Flip used | Step after the flip | Energy (sample) |
|---|---|---|---|---|
| ↘ | (+1, +1) | none | ↘ (1, 1) | 32 |
| ↙ | (+1, −1) | reverse each row (mirror left–right) | ↘ (1, 1) | 32 |
| ← | (0, −2) | reverse each row | → (0, 2) | 29 |
| ↖ | (−1, −1) | reverse each row and the row order | ↘ (1, 1) | 32 |
| ↗ | (−1, +1) | reverse the row order (mirror top–bottom) | ↘ (1, 1) | 30 |
| → | (0, +2) | none | → (0, 2) | 32 |

**Step-by-step solution**

1. Turn Part I into a function that takes a look-back step: (1 row, 1 column) for diagonals, (0 rows, 2 columns) for horizontals.
2. For each of the six directions, mirror a fresh copy of the field so that the direction becomes ↘ or →.
3. Run the function on each copy and add the six counts.

## My solution

```ts
const getEnergyRequired = (
  inputs: string[][],
  top: number = 1,
  left: number = 1,
): number => {
  const heights: string[][] = (
    JSON.parse(JSON.stringify(inputs)) as string[][]
  ).map((i) => i.fill("."));

  const isOutside = (row: number, col: number) => {
    const prevRow = row - top;
    const prevCol = col - left;

    return (
      prevRow < 0 ||
      prevRow >= inputs.length ||
      prevCol < 0 ||
      prevCol >= inputs[0]!.length
    );
  };

  const getHeight = (row: number, col: number) => {
    if (inputs[row - top]![col - left] === ".") return inputs[row]![col] || ".";

    const currentHeight = Number(inputs[row]![col]);
    const prevHeight = Number(heights[row - top]![col - left]);
    let height = 0;

    if (currentHeight > prevHeight - 1) height = currentHeight;
    else height = prevHeight - 1;

    return String(height);
  };
```

q8/part2.ts, lines 10–42 (highlighted lines are the ones that changed from the previous part)

```
  for (let row = 0; row < inputs.length; row++) {
    for (let col = 0; col < inputs[0]!.length; col++) {
      if (inputs[row]![col] === ".") continue;

      if (isOutside(row, col)) heights[row]![col] = inputs[row]![col] || ".";
      else heights[row]![col] = getHeight(row, col);
    }
  }

  for (let row = 0; row < inputs.length; row++) {
    for (let col = 0; col < inputs[0]!.length; col++) {
      if (
        isOutside(row, col) ||
        inputs[row]![col] === "." ||
        heights[row - top]![col - left] === "."
      )
        continue;

      if (Number(inputs[row]![col]) < Number(heights[row - top]![col - left]))
        inputs[row]![col] = ".";
    }
  }

  let energyRequired = 0;

  inputs.forEach((row) => {
    row.forEach((col) => {
      if (col !== ".") energyRequired++;
    });
  });

  return energyRequired;
};
```

q8/part2.ts, lines 44–76 (highlighted lines are the ones that changed from the previous part)

| Lines | What happens |
|---|---|
| 10–14 | Part I's code becomes a function `getEnergyRequired(inputs, top, left)`. The previous hex is (row − `top`, col − `left`); the defaults 1, 1 mean ↘ like Part I. |
| 15–17 | Same deep copy filled with dots as before. |
| 19–29 | `isOutside` replaces Part I's `row === 0 \|\| col === 0`: the previous hex is off the grid. With `left = 2` this correctly treats columns 0 *and* 1 as the start of a line. |
| 31–42 | `getHeight` as before, but looking back by (`top`, `left`). |
| 44–65 | The same two passes as Part I: skyline first, then shade the Ents below the previous skyline. |
| 67–75 | Count and **return** instead of printing. |

```ts
let totalEnergy = 0;

// Direction: ↘
totalEnergy += getEnergyRequired(JSON.parse(JSON.stringify(INPUTS)));

// Direction: ↙
totalEnergy += getEnergyRequired(
  (JSON.parse(JSON.stringify(INPUTS)) as string[][]).map((i) => i.reverse()),
);

// Direction: ←
totalEnergy += getEnergyRequired(
  (JSON.parse(JSON.stringify(INPUTS)) as string[][]).map((i) => i.reverse()),
  0,
  2,
);

// Direction: ↖
totalEnergy += getEnergyRequired(
  (JSON.parse(JSON.stringify(INPUTS)) as string[][])
    .map((i) => i.reverse())
    .reverse(),
);

// Direction: ↗
totalEnergy += getEnergyRequired(
  (JSON.parse(JSON.stringify(INPUTS)) as string[][]).reverse(),
);

// Direction: →
totalEnergy += getEnergyRequired(JSON.parse(JSON.stringify(INPUTS)), 0, 2);

console.log(totalEnergy);
```

q8/part2.ts, lines 78–110 (highlighted lines are the ones that changed from the previous part)

| Lines | What happens |
|---|---|
| 81 | ↘: a copy of the field as it is. |
| 84–86 | ↙: reverse each row. `reverse()` works in place, but it's on a fresh deep copy each time, so `INPUTS` is never changed. |
| 89–93 | ←: reverse each row, then look back two columns (`0, 2`). |
| 96–100 | ↖: reverse each row and the order of the rows (a 180° turn). |
| 103–105 | ↗: reverse only the order of the rows. |
| 108 | →: the original field, looking back two columns. |

### Trace: the six rounds on the sample (from an instrumented copy)

Each call returned 32, 32, 29, 32, 30, 32, and the shaded grids it produced are the puzzle's six maps, just drawn mirrored for the flipped rounds. For example, the ← round works on the left–right mirror of the field and its lit Ents, mirrored back, are exactly the 29 shown in the puzzle's round 3 map. Total: **187**.

> **Why ← and → need the `(0, 2)` step:** you could also flip ← into ↘ by turning the grid, but on a hex grid drawn as text a 60° turn isn't a simple array operation. Mirrors are, and the six directions split into exactly two classes under mirroring: the four diagonals and the two horizontals. Hence one function with two settings.

## 5. Part III: the growing field

### The question

Now the field changes after every round, and the source keeps cycling ↘, ↙, ←, ↖, ↗, → , ↘, …

1. First, all shadows for the round are worked out on the field as it is.
2. Every lit Ent uses one unit of energy and grows by 1.
3. A lit Ent of height 9 becomes a warrior instead. It leaves a **seed**, which stays underground for the next 3 rounds (no energy, no shadow) and sprouts as a height-1 Ent at the start of the round after that.
4. Stop after the round in which the total number of warriors first reaches 100,000, and report the total energy of all rounds.

A seed has no height, but shadows still **pass over** it and keep shrinking by 1 per hex, as over any other hex.

### Worked example (sample)

The puzzle shows the first ten rounds; your code (instrumented) gives exactly the same numbers:

| Round | Direction | Energy | New warriors | Warriors so far | Energy so far |
|---|---|---|---|---|---|
| 1 | ↘ | 32 | 8 | 8 | 32 |
| 2 | ↙ | 31 | 6 | 14 | 63 |
| 3 | ← | 29 | 5 | 19 | 92 |
| 4 | ↖ | 22 | 4 | 23 | 114 |
| 5 | ↗ | 25 | 5 | 28 | 139 |
| 6 | → | 27 | 1 | 29 | 166 |
| 7 | ↘ | 28 | 8 | 37 | 194 |
| 8 | ↙ | 27 | 1 | 38 | 221 |
| 9 | ← | 31 | 6 | 44 | 252 |
| 10 | ↖ | 26 | 1 | 45 | 278 |

The army reaches 100,000 warriors in round **32008**, and the total energy is **899889**, both matching the puzzle.

**Extra examples of my own (run through a copy of the real code with a smaller warrior target)**

| Field | Target | Rounds | Energy | What happens |
|---|---|---|---|---|
| a single `9` | 2 | 13 | 10 | Round 1: lit, first warrior. Rounds 2–4: seed underground, energy 0. Round 5: sprouts as 1 and is lit (it's alone), grows 1 → 9 over rounds 5–12, and is lit as a 9 in round 13: second warrior. 1 + 9 = 10. |
| `9.1` (two Ents side by side) | 2 | 9 | 14 | The 9 leaves in round 1. It sprouts in round 5; in round 6 (→) the left Ent (2) casts a shadow of only 1 on the right one (6), so both are lit. In round 9 (←) the right Ent is a 9, lit first; its shadow of 8 hides the left 5. Energy per round: 2, 1, 1, 1, 2, 2, 2, 2, 1. |
| `6.x.3` (→, one round) | | | 1 | The shadow of 6 crosses the seed as 5 and reaches the 3 as 4: the 3 is shaded. |
| `6.x.6` (→, one round) | | | 2 | Same shadow, but 6 > 4, so it is lit. |

**Step-by-step solution**

1. Keep the field between rounds; store seeds as countdowns `x4` … `x1`.
2. At the start of each round, count every seed down by one, turning it into a height-1 Ent when its countdown runs out.
3. Flip the field into the right orientation for this round's direction and count the lit Ents, treating seeds as height 0 and never lit.
4. Grow every lit Ent by 1; a lit 9 becomes a new warrior and a seed `x4`.
5. Add the round's energy and warriors; repeat until there are at least 100,000 warriors, then print the total energy.

## My solution

Lines 1–8 are the shared code (with `sample3.txt` / `input3.txt`; `INPUTS` is now `let`, because the field is replaced every round).

```ts
const MIN_ENT = 100000;

const getEnergyRequired = (
  inputs: string[][],
  top: number = 1,
  left: number = 1,
) => {
  const heights: string[][] = (
    JSON.parse(JSON.stringify(inputs)) as string[][]
  ).map((i) => i.fill("."));

  const isOutside = (row: number, col: number) => {
    const prevRow = row - top;
    const prevCol = col - left;

    return (
      prevRow < 0 ||
      prevRow >= inputs.length ||
      prevCol < 0 ||
      prevCol >= inputs[0]!.length
    );
  };

  const getHeight = (row: number, col: number) => {
    const currentHeight =
      inputs[row]![col] === "x" ? 0 : Number(inputs[row]![col]);

    if (inputs[row - top]![col - left] === ".") return String(currentHeight);

    const prevHeight = Number(heights[row - top]![col - left]);
    let height = 0;

    if (currentHeight > prevHeight - 1) height = currentHeight;
    else height = prevHeight - 1;

    return String(height);
  };
```

q8/part3.ts, lines 10–46 (highlighted lines are the ones that changed from the previous part)

```
48    for (let row = 0; row < inputs.length; row++) {
49      for (let col = 0; col < inputs[0]!.length; col++) {
50        if (inputs[row]![col] === ".") continue;
51
52        if (isOutside(row, col))
53          heights[row]![col] =
54            inputs[row]![col] === "x" ? "0" : inputs[row]![col]!;
55        else heights[row]![col] = getHeight(row, col);
56      }
57    }
58
59    for (let row = 0; row < inputs.length; row++) {
60      for (let col = 0; col < inputs[0]!.length; col++) {
61        if (
62          isOutside(row, col) ||
63          inputs[row]![col] === "." ||
64          inputs[row]![col] === "x" ||
65          heights[row - top]![col - left] === "." ||
66          heights[row - top]![col - left] === "x"
67        )
68          continue;
69
70        if (Number(inputs[row]![col]) < Number(heights[row - top]![col - left]))
71          inputs[row]![col] = ".";
72      }
73    }
74
75    let energyRequired = 0;
76    let newWarriors = 0;
77
78    inputs.forEach((row) => {
79      row.forEach((col) => {
80        if (col !== "." && col !== "x") energyRequired++;
81        if (col === "9") newWarriors++;
82      });
83    });
84
85    return { energyRequired, newWarriors, inputs };
86  };
```

q8/part3.ts, lines 48–86 (highlighted lines are the ones that changed from the previous part)

| Lines | What happens |
|---|---|
| 10 | The warrior target, the same for the sample and the real input. |
| 12–16 | The same `getEnergyRequired(inputs, top, left)` as Part II, but its grid can now also contain `"x"` for a seed. |
| 33–37 | A seed counts as height 0, so its skyline is just the shadow passing over it: `max(0, prev - 1)`. That's how a shadow crosses a seed and keeps shrinking. |
| 52–54 | A seed at the start of a line gets skyline 0: it casts nothing. |
| 61–72 | Second pass: seeds are skipped (line 64), they need no energy and can't be shaded. Line 66 can never be true, because `heights` stores `"0"` for seeds, never `"x"`. It's harmless. |
| 75–85 | Count lit Ents (not dots, not seeds), and lit 9s as new warriors. Also return the shaded grid, so the caller knows which Ents to grow. |

```ts
const sprout = (field: string[][]) =>
  field.map((row) =>
    row.map((cell) => {
      if (!cell.startsWith("x")) return cell;
      const left = Number(cell.slice(1)) - 1;
      return left === 0 ? "1" : `x${left}`;
    }),
  );

const seedsAsX = (field: string[][]) =>
  field.map((row) => row.map((cell) => (cell.startsWith("x") ? "x" : cell)));

let totalEnergy = 0;
let totalEnt = 0;
let i = 1;

while (totalEnt < MIN_ENT) {
  INPUTS = sprout(INPUTS);

  let result = {
    energyRequired: 0,
    newWarriors: 0,
    inputs: INPUTS,
  };
```

q8/part3.ts, lines 88–111 (highlighted lines are the ones that changed from the previous part)

```
  switch ((i - 1) % 6) {
    case 0:
      // Direction: ↘
      result = getEnergyRequired(seedsAsX(INPUTS));
      break;
    case 1:
      // Direction: ↙
      INPUTS = (JSON.parse(JSON.stringify(INPUTS)) as string[][]).map((i) =>
        i.reverse(),
      );
      result = getEnergyRequired(seedsAsX(INPUTS));
      break;
    case 2:
      // Direction: ←
      result = getEnergyRequired(seedsAsX(INPUTS), 0, 2);
      break;
    case 3:
      // Direction: ↖
      INPUTS = (JSON.parse(JSON.stringify(INPUTS)) as string[][]).reverse();
      result = getEnergyRequired(seedsAsX(INPUTS));
      break;
    case 4:
      // Direction: ↗
      INPUTS = (JSON.parse(JSON.stringify(INPUTS)) as string[][]).map((i) =>
        i.reverse(),
      );
      result = getEnergyRequired(seedsAsX(INPUTS));
      break;
    case 5:
      // Direction: →
      INPUTS = (JSON.parse(JSON.stringify(INPUTS)) as string[][]).reverse();
      result = getEnergyRequired(seedsAsX(INPUTS), 0, 2);
      break;
  }

  result.inputs.forEach((row, rowIdx) =>
    row.forEach((col, colIdx) => {
      if (col === "." || col === "x") return;

      if (col === "9") INPUTS[rowIdx]![colIdx] = "x4";
      else
        INPUTS[rowIdx]![colIdx] = String(Number(INPUTS[rowIdx]![colIdx]) + 1);
    }),
  );

  totalEnergy += result.energyRequired;
  totalEnt += result.newWarriors;
  i++;
}

console.log(totalEnergy);
```

q8/part3.ts, lines 113–163 (highlighted lines are the ones that changed from the previous part)

| Lines | What happens |
|---|---|
| 88–95 | `sprout` runs at the start of every round. A seed is stored as `"x4"`, `"x3"`, `"x2"`, `"x1"`, a countdown. Each round it counts down by 1, and when it would reach 0 it becomes `"1"`. |
| 97–98 | `seedsAsX` makes a copy where every `"xN"` is just `"x"`, which is what `getEnergyRequired` expects. |
| 104 | Keep going until the army is big enough. The check is at the top, so the round that reaches 100,000 is still counted. |
| 113–146 | Pick the direction by round number, 0 to 5. Unlike Part II, the flips are applied to `INPUTS` itself and **stay**: each round flips the field from *last round's* orientation. ↙ mirrors left–right; ← keeps that mirror and uses `(0, 2)`; ↖ also mirrors top–bottom (now both); ↗ undoes the left–right mirror (only top–bottom left); → undoes the top–bottom one (back to the original). So after round 6 the field is the right way round again, ready for ↘. |
| 148–156 | Growth, using the shaded grid from this round, which has the same orientation as `INPUTS`. Every lit cell either becomes a seed `"x4"` (if it was a 9) or grows by 1. The shadows were all decided before this loop, as the rules require. |
| 158–160 | Add up the energy and the warriors, and go to the next round. |

### Trace: a seed's countdown

| Round | Cell at the start of the round (after `sprout`) | Energy from this cell |
|---|---|---|
| r | `9`, lit, becomes `x4` on line 152 | 1 |
| r+1 | `x3` (underground) | 0 |
| r+2 | `x2` (underground) | 0 |
| r+3 | `x1` (underground) | 0 |
| r+4 | `1` (sprouted) | 1 if lit |

Starting at `x4` instead of `x3` is what makes the timing work: `sprout` runs *before* the round, so a fresh seed has already counted down once when its first underground round starts. You can see this in the puzzle's round 2 map, where the Ents that left in round 1 are drawn as `X`, and in round 5, where they are back as 1s.

> **Fragile: the orientation is carried from round to round.** This is correct, and the sample matches all ten rounds and the final 899889. But each case only works because the previous case left the field in a specific flip, so the six cases can't be reordered, skipped or tested on their own. If the loop ever started at a different round, or one case were changed, every later round would be lit from the wrong side, and the totals could still look plausible. Part II's approach (flip a fresh copy from the original each time) avoids this, at the cost of flipping the growth back afterwards.

> **Small things:** the callbacks on lines 120 and 136 call their parameter `i`, which hides the round counter `i` inside them (harmless, but easy to trip over). Line 66 is dead code, as noted above. **Cost:** 32008 rounds, each making a few `JSON` deep copies of the grid; the whole run takes about half a second on the sample and on the real input.

## 6. Summary

| Part | Directions | Field changes? | Key trick in your code | Sample |
|---|---|---|---|---|
| I | ↘ only | no | a skyline grid (Ent or shadow, whichever is higher), then compare each Ent to the previous skyline | 32 |
| II | all six, once | no | one function with a look-back step `(1,1)` or `(0,2)`, and mirrored copies of the field | 187 |
| III | all six, cycling | yes: growth, warriors, 3-round seeds | seeds as `"xN"` countdowns, seeds at height 0, and flips carried between rounds | 899889 |

**Key ideas:** a shadow that drops by 1 per step is a **running maximum** ("skyline") along each line; a hex grid stored as text has two kinds of straight lines (diagonals and horizontals), and **mirroring** turns every direction into one of them; and in a simulation, decide everything for the round first (shadows), then apply the changes (growth).